Department of Computer Science and Information Technology

CSCI426: Advanced Web Programming

MovieNest

Project Phase 1 Report

Prepared by

Sami Zorkot
Issa Hammad

Fall 2026

GitHub: https://github.com/Sami-CSCI/movienest

Live website: https://sami-csci.github.io/movienest/

Abstract

Choosing a movie often takes longer than watching one. People scroll through long lists on streaming apps and forget the titles friends recommended to them. MovieNest is a small, responsive frontend web application built with ReactJS that solves this everyday problem. The user can browse a curated list of movies, search them by title, filter them by genre, open a dedicated page for every movie, and save movies to a personal favourites list. The favourites list is stored as JSON in the browser's localStorage, so it is still there after the page is refreshed or the browser is closed, without any account or server. The application has seven routes (Home, Movies, Movie Details, Features, Favourites, About and Contact), uses Tailwind CSS and Untitled UI React components for a clean interface, is managed with Git and is deployed on GitHub Pages.

1. Introduction

1.1 Problem

Most movie platforms are made to keep users scrolling. When someone just wants to remember a few movies to watch later, those platforms are too heavy, they need an account, and the list is tied to one service. A simple personal tool that works in any browser is missing.

1.2 Solution

MovieNest is a light website that shows a short list of well known movies with their genre, year, rating and a short summary, and lets the user build a favourites list. Everything runs in the browser: there is no backend and no sign up.

1.3 Objectives

- Apply web design and development principles using ReactJS as a frontend.
- Build a responsive interface that works on desktop and mobile.
- Use JSON and localStorage to keep user data in the browser.
- Use Git for version control and deploy the project on GitHub Pages.

2. System Design

2.1 Architecture

MovieNest is a single page application (SPA). The browser downloads one HTML page and the JavaScript bundle once. After that, React Router swaps the page components when the user clicks a link, without reloading the page. The application has three layers:

Layer	Files	Role
Presentation	src/pages/*.jsx, src/components/*.jsx	Pages and reusable components that build the user interface with JSX and Tailwind CSS.
Routing	src/main.jsx, src/App.jsx	HashRouter and Routes decide which page component is shown for each URL.
Data	src/data/movies.js, src/favourites.js	A static array of movie objects, and helper functions that read and write the favourites array as JSON in localStorage.

2.2 Component tree

```
main.jsx
 └── HashRouter
      └── App
           ├── Navbar                (links to every page, mobile menu)
           ├── Routes
           │    ├── /              → Home         (uses MovieCard, Button, Badge)
           │    ├── /movies        → Movies       (uses PageHeader, MovieCard, Button)
           │    ├── /movies/:id    → MovieDetails (uses Poster, Badge, Button)
           │    ├── /features      → Features     (uses PageHeader)
           │    ├── /favourites    → Favourites   (uses Button, favourites.js)
           │    ├── /about         → About        (uses PageHeader, Badge)
           │    └── /contact       → Contact      (uses PageHeader, Button)
           └── Footer
```

2.3 Pages and routes

Page	Route	Description
Home	/	Hero section with call-to-action buttons and the four top rated movies.
Movies	/movies	All movies in a responsive grid, with a search box and genre filter buttons.
Movie Details (dynamic)	/movies/:id	Reads the id from the URL with useParams and shows that movie. The same component renders a different movie for every id. The user can add the movie to favourites.
Features (Services)	/features	Cards describing what the website offers.
Favourites	/favourites	Form to add a favourite, list of saved favourites with a delete button. Saved in localStorage.
About	/about	Why the project was made, technologies and the team.
Contact	/contact	Contact form with validation and a success message.

2.4 Data model

Each movie in `src/data/movies.js` is a JavaScript object:

```
  {
    id: 1,
    title: "Inception",
    year: 2010,
    genre: "Sci-Fi",
    rating: 8.8,
    duration: "2h 28m",
    director: "Christopher Nolan",
    colors: ["#1e3a8a", "#0ea5e9"],
    description:
      "A thief who steals secrets through dreams is given one last job: plant an idea in someone's
          mind instead of taking one.",
  },
```

Each favourite saved in localStorage under the key `react-favourites` is a smaller object. The whole list is stored as one JSON string:

```
[{"id":1794129643404,"title":"Inception","genre":"Sci-Fi"},
 {"id":1794129644404,"title":"Up","genre":"Animation"}]
```

2.5 User flow

- The user opens the Home page and sees the top rated movies.
- They go to Movies, type in the search box or click a genre to filter the grid.
- Clicking a movie opens /movies/:id with its details.
- Clicking *Add to favourites* saves it; the Favourites page lists it, and the user can also add movies by hand or delete them.
- Refreshing the browser keeps the list, because it is loaded from localStorage on start.

2.6 Responsive design and UI/UX

The layout is mobile-first with Tailwind breakpoints. The movie grid shows 2 columns on phones, 3 on tablets and 4 on laptops (`grid-cols-2 sm:grid-cols-3 lg:grid-cols-4`). The navigation bar shows all links on wide screens and turns into a menu button (☰) on small screens. The details and contact pages are one column on phones and two columns on wider screens. Colours, spacing and typography come from the Untitled UI theme, so every page looks consistent. Active links are highlighted, buttons have hover and focus states and form errors appear under each field.

3. Technologies Used

Technology	Use in the project
ReactJS 19	Building the interface with function components, props and the useState hook.
Vite	Development server and production build tool.
React Router (react-router-dom)	Client-side routing: HashRouter, Routes, Route, Link, NavLink and useParams.
Tailwind CSS 4	Utility classes for layout, spacing, colours and responsive breakpoints.
Untitled UI React	Ready Button and Badge components (built on React Aria) and the design theme.
JSON and localStorage	Saving and loading the favourites list in the browser.
Git and GitHub	Version control and hosting of the source code.
GitHub Pages and gh-pages	Hosting the built website for free.

4. Code Snippets (key parts)

4.1 Routing (App.jsx)

All pages are registered in one place. `:id` makes the details route dynamic.

```
function App() {
  return (
    <div className="flex min-h-screen flex-col bg-primary">
      <Navbar />

      <main className="mx-auto w-full max-w-6xl flex-1 px-4 py-10 md:px-8">
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/movies" element={<Movies />} />
          <Route path="/movies/:id" element={<MovieDetails />} />
          <Route path="/features" element={<Features />} />
          <Route path="/favourites" element={<Favourites />} />
          <Route path="/about" element={<About />} />
          <Route path="/contact" element={<Contact />} />
        </Routes>
      </main>

      <Footer />
    </div>
  );
}

export default App;
```

4.2 Saving favourites as JSON (favourites.js)

These helpers come from the course tutorial. `JSON.stringify` turns the array into text before saving, and `JSON.parse` turns it back into an array.

```
const STORAGE_KEY = "react-favourites";

export function loadFavourites() {
  try {
    const savedText = localStorage.getItem(STORAGE_KEY);

    if (savedText === null) {
      return [];
    }

    const parsedData = JSON.parse(savedText);

    if (!Array.isArray(parsedData)) {
      return [];
    }

    return parsedData;
  } catch (error) {
    console.error("Unable to load favourites:", error);
    return [];
  }
}

export function saveFavourites(favourites) {
  localStorage.setItem(
    STORAGE_KEY,
    JSON.stringify(favourites)
  );
}
```

4.3 Adding and deleting a favourite (Favourites.jsx)

```
const [favourites, setFavourites] = useState(
  () => loadFavourites()
);

const [title, setTitle] = useState("");
const [genre, setGenre] = useState("");

function handleAddFavourite(event) {
  event.preventDefault();

  if (title.trim() === "" || genre.trim() === "") {
    alert("Please enter both a movie title and a genre.");
    return;
  }

  const newFavourite = {
    id: Date.now(),
    title: title.trim(),
    genre: genre.trim()
  };

  const updatedFavourites = [
    ...favourites,
    newFavourite
  ];

  setFavourites(updatedFavourites);
  saveFavourites(updatedFavourites);

  setTitle("");
  setGenre("");
}

function handleDeleteFavourite(id) {
  const updatedFavourites = favourites.filter(
    (item) => item.id !== id
  );

  setFavourites(updatedFavourites);
  saveFavourites(updatedFavourites);
}
```

4.4 The dynamic page (MovieDetails.jsx)

```
function MovieDetails() {
  const { id } = useParams();
  const movie = movies.find((item) => item.id === Number(id));

  const [saved, setSaved] = useState(
    () => loadFavourites().some((item) => item.title === movie?.title)
  );

  if (!movie) {
    return (
      <div className="py-20 text-center">
        <h1 className="text-2xl font-bold text-primary">Movie not found</h1>
        <Button href="#/movies" className="mt-6">Back to movies</Button>
      </div>
    );
  }

  function handleSave() {
    const favourites = loadFavourites();
    const newFavourite = {
      id: Date.now(),
      title: movie.title,
      genre: movie.genre
    };

    saveFavourites([...favourites, newFavourite]);
    setSaved(true);
  }
```

4.5 Search and genre filter (Movies.jsx)

```
function Movies() {
  const [search, setSearch] = useState("");
  const [genre, setGenre] = useState("All");

  const filteredMovies = movies.filter((movie) => {
    const matchesSearch = movie.title.toLowerCase().includes(search.toLowerCase());
    const matchesGenre = genre === "All" || movie.genre === genre;
    return matchesSearch && matchesGenre;
  });
```

4.6 Using an Untitled UI component

```
        <div className="flex flex-wrap gap-2">
          {genres.map((item) => (
            <Button
              key={item}
              size="sm"
              color={genre === item ? "primary" : "secondary"}
              onClick={() => setGenre(item)}
            >
              {item}
            </Button>
```

4.7 Contact form validation (Contact.jsx)

```
const handleSubmit = (e) => {
  e.preventDefault();
  let validationErrors = {};

  if (name.trim() === "") validationErrors.name = "Name is required";
  if (email.trim() === "") validationErrors.email = "Email is required";
  else if (!email.includes("@")) validationErrors.email = "Enter a valid email";
  if (subject.trim() === "") validationErrors.subject = "Subject is required";
  if (message.trim() === "") validationErrors.message = "Message is required";

  if (Object.keys(validationErrors).length > 0) {
    setErrors(validationErrors);
    setSuccess("");
  } else {
    const formData = { name, email, subject, message };
    console.log(formData);
    setSuccess("Your message has been sent successfully!");
    setErrors({});
    setName("");
    setEmail("");
    setSubject("");
    setMessage("");
  }
};
```

5. User Interface

The following screenshots were taken on a laptop screen (1280px wide) and on a phone screen (390px wide).

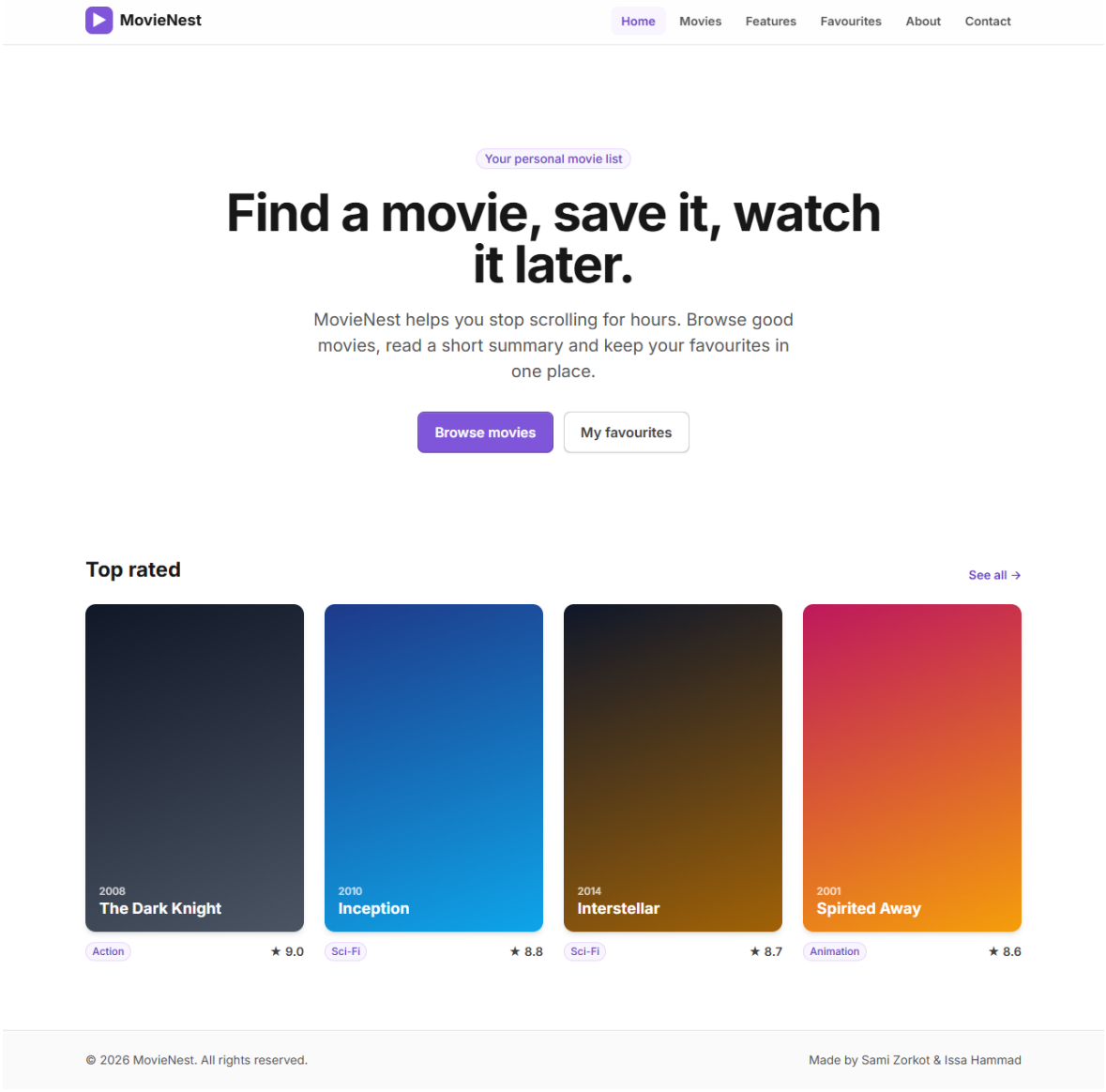


Figure 1: Home page

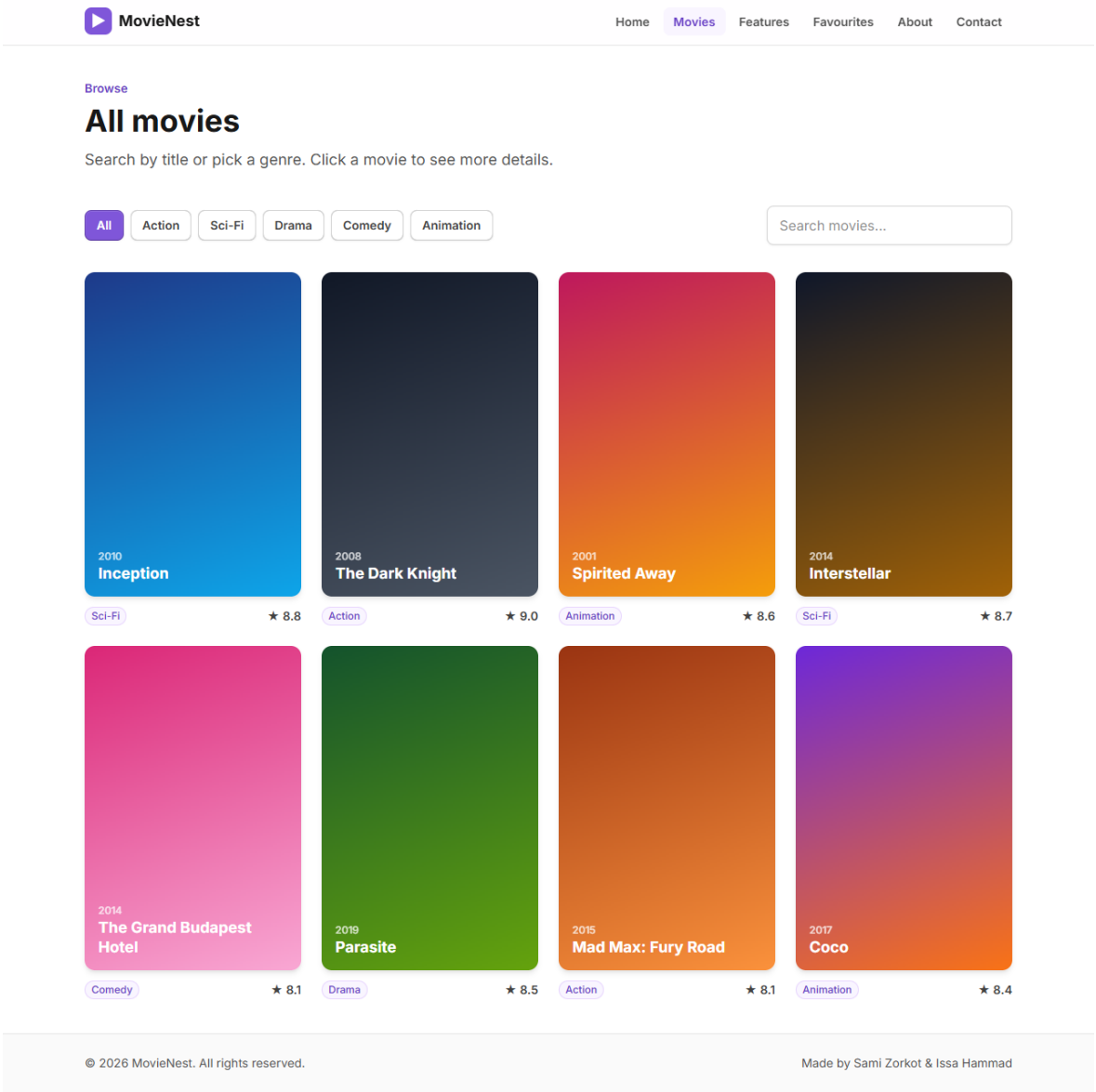


Figure 2: Movies page with search and genre filter

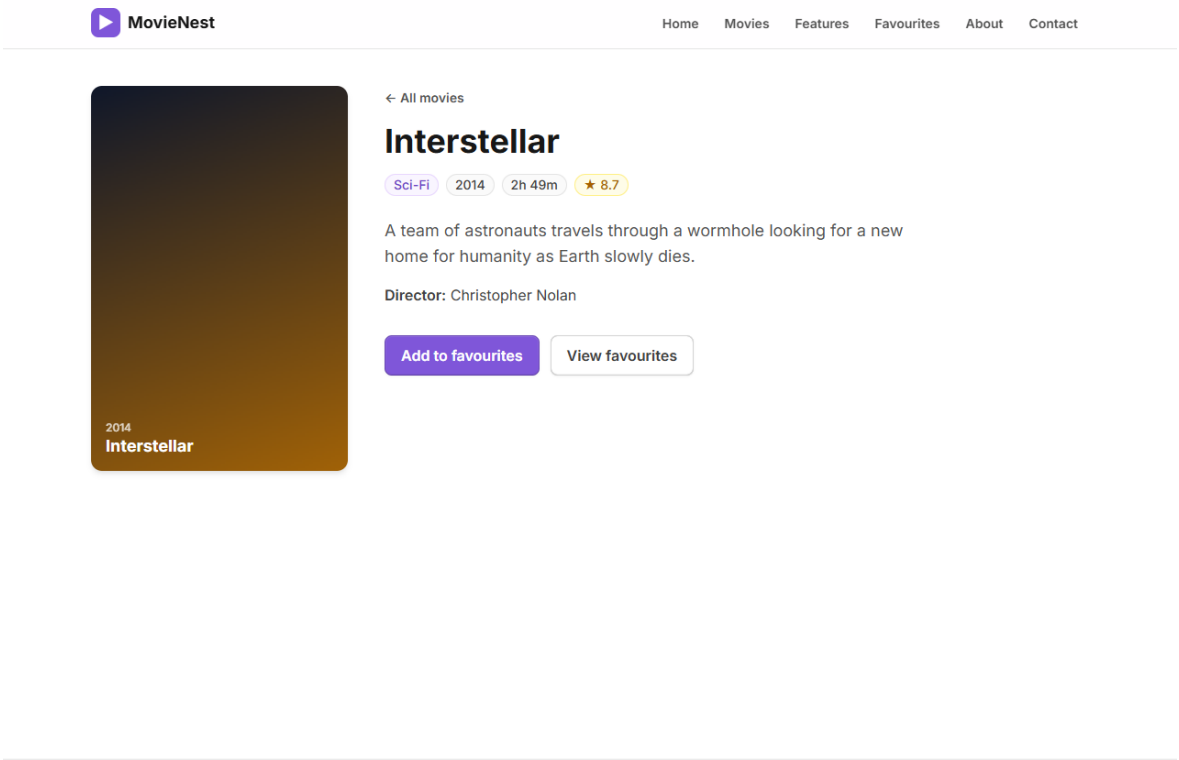


Figure 3: Movie details (dynamic page)

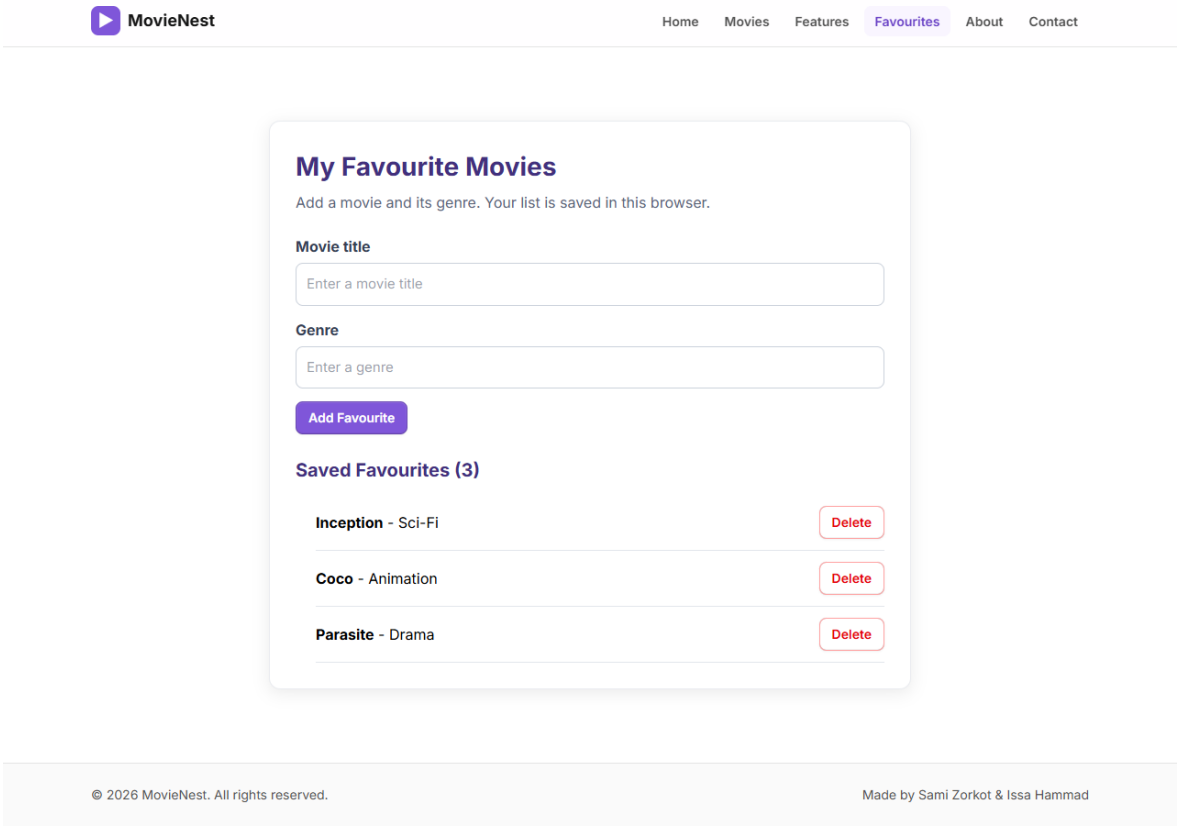


Figure 4: Favourites page (JSON + localStorage)

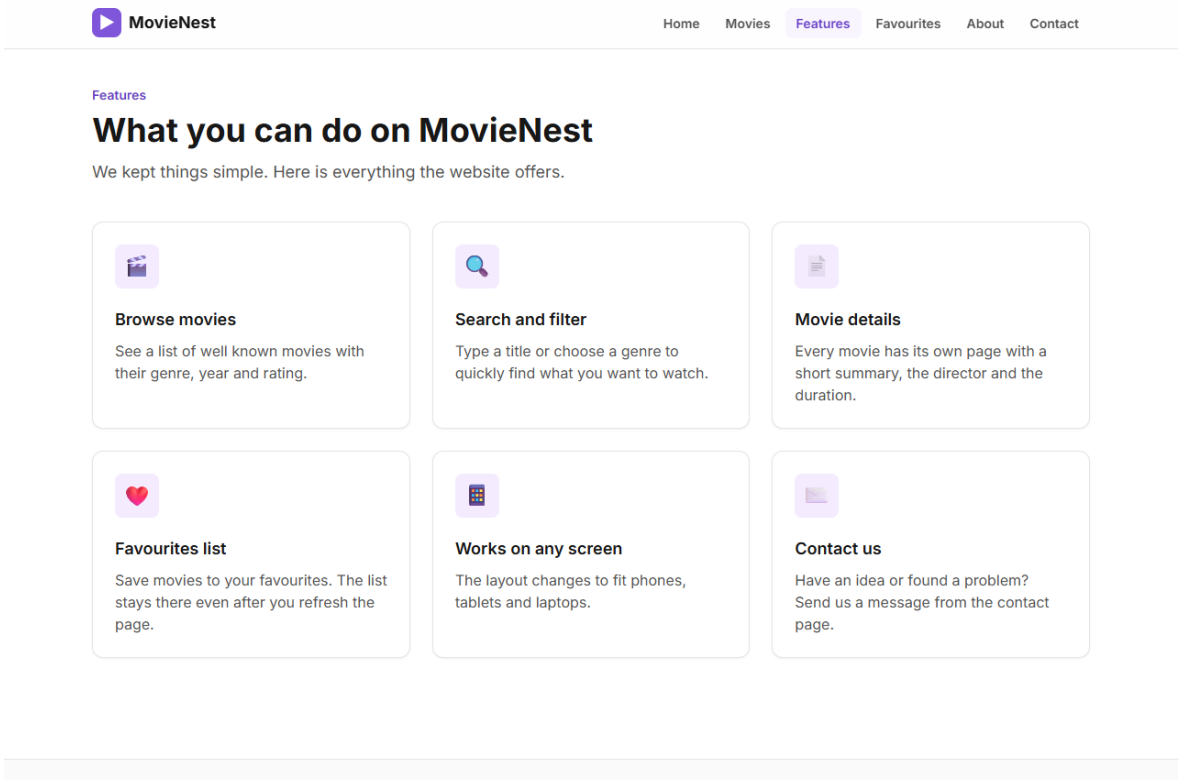


Figure 5: Features page

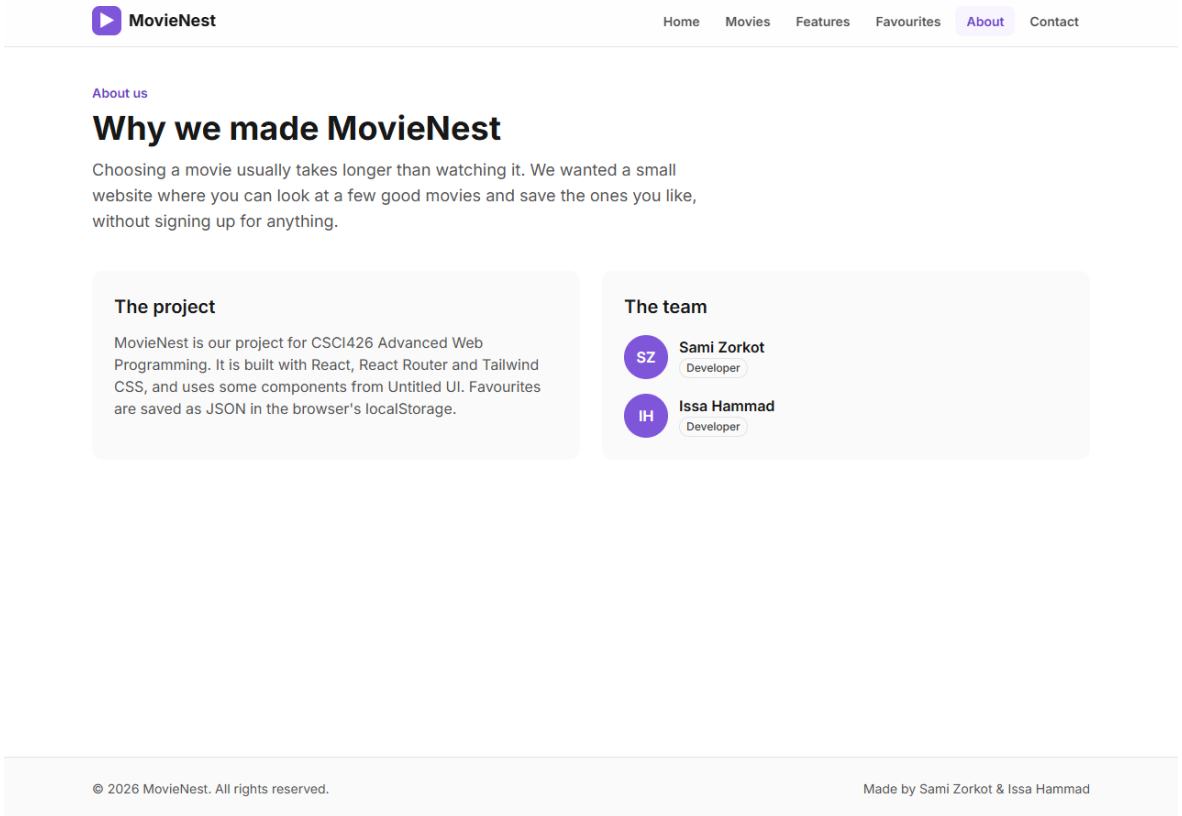


Figure 6: About page

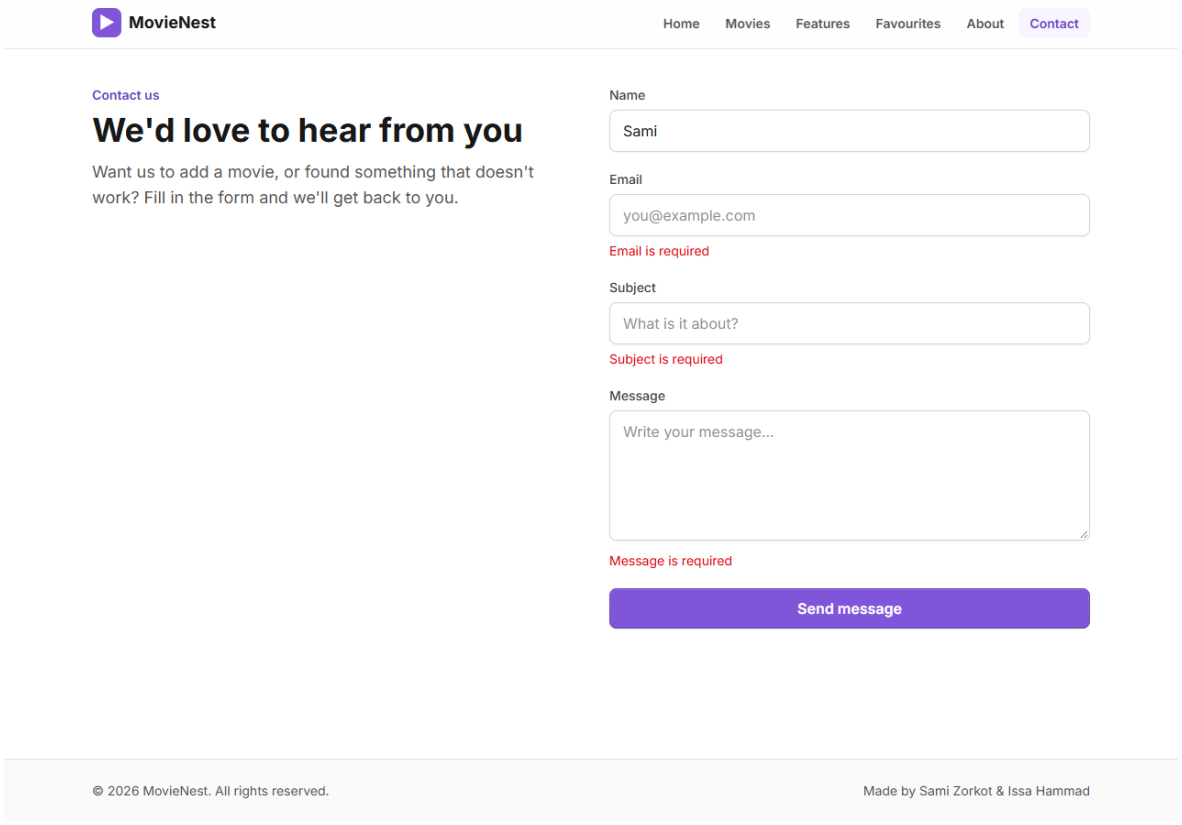


Figure 7: Contact page showing validation errors

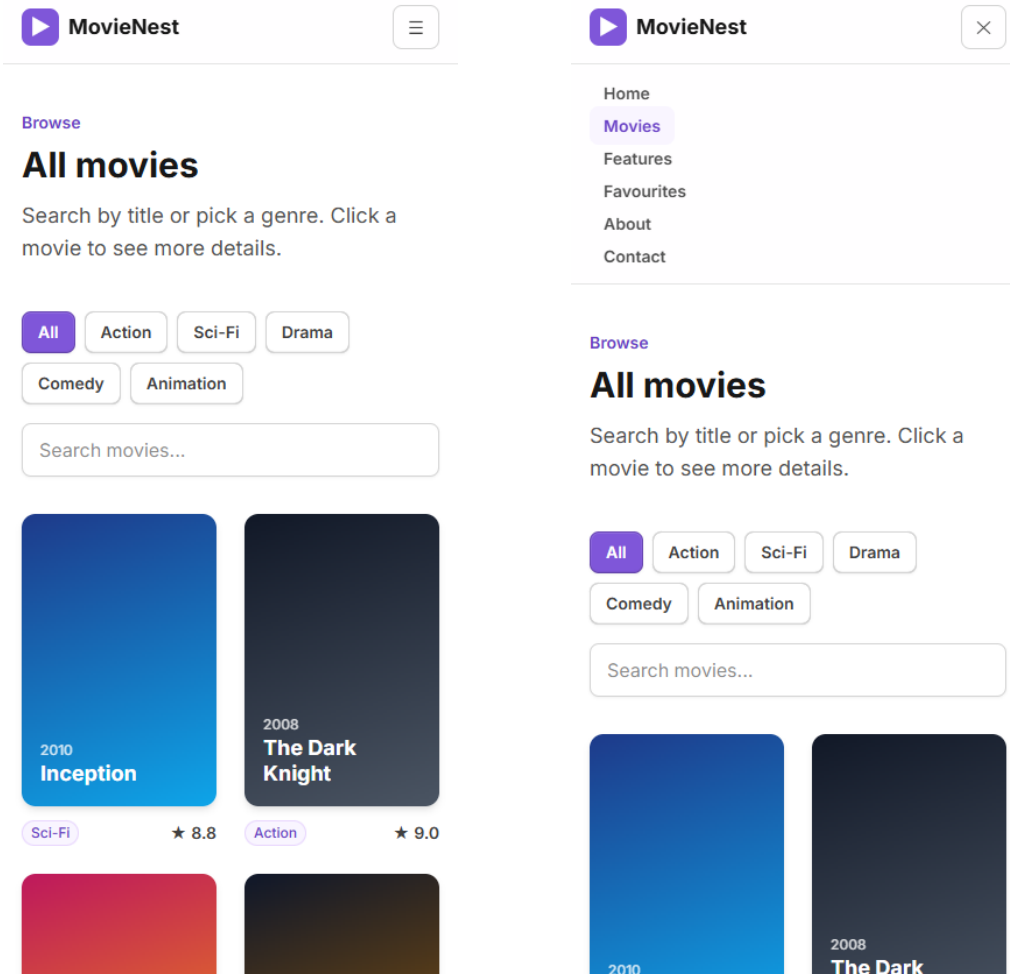


Figure 8: Mobile view, with the menu closed and open

6. Version Control and Deployment

The project is tracked with Git and pushed to the GitHub repository `Sami-CSCI/movienest`, where the commit history can be seen. The website is built with `vite build` and published to the `gh-pages` branch with the `gh-pages` package (`npm run deploy`). GitHub Pages then serves it at https://sami-csci.github.io/movienest/. HashRouter is used so that refreshing any page works on GitHub Pages, which only serves static files.

7. Group Contribution Statement

Both members worked on the project together. The table shows the main responsibilities of each member.

Member	Main contributions	Share
Sami Zorkot	Project setup (Vite, Tailwind, Untitled UI), routing, Navbar and Footer, Home, Movies and Movie Details pages, GitHub repository and deployment.	50%
Issa Hammad	Favourites page with JSON and localStorage, Features, About and Contact pages, form validation, README, screenshots and this report.	50%

8. Conclusion

MovieNest meets the Phase 1 requirements: it is a ReactJS frontend with more than five pages including a dynamic one, it is responsive, it uses Tailwind CSS, it saves user data with JSON and localStorage, it is managed with Git and it is hosted on GitHub Pages. A later phase could add a backend so the favourites follow the user across devices, and real movie posters from a movie API.

9. References and Credits

- Untitled UI React components (MIT license): https://www.untitledui.com/react/components. The files in src/components/base, src/styles and src/utils were added with the untitledui CLI.
- Course tutorial: *JSON, React and Local Storage*, section 8, Building the React Favourites Application.
- Course tutorials on React components, React Router and React Forms.
- React documentation: https://react.dev
- React Router documentation: https://reactrouter.com
- Tailwind CSS documentation: https://tailwindcss.com